



BITS Pilani

MFML AND MFDS

MFML and MFDS Team
WILPD - CSIS



* ZC416 Mathematical Foundations for
Machine Learning / Data Science

Topic: # 11 Optimization

DIFFICULT TOPOLOGIES - CLIFFS



- Two examples of high curvature surfaces are cliffs and valleys.
- An example of a cliff is shown below:

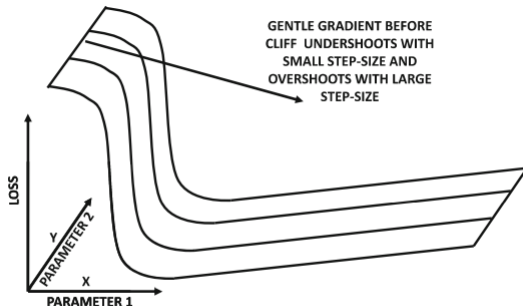


Figure 5.3: An example of a cliff in the loss surface

DIFFICULT TOPOLOGIES - CLIFFS



- Considering the previous figure, we see that the partial derivative with respect to x changes drastically as we go along the axis of x .
- A modest learning rate will cause minimal reduction in the value of the objective function in the gently sloping regions.
- The same modest learning rate in the high-sloping regions will cause us to overshoot the optimal value in those regions.
- The problem is caused by the nature of curvature - the first order gradient does not have any information that will help control the size of the update.
- We need to look at second-order derivatives in this case.

DIFFICULT TOPOLOGIES - VALLEYS



- Consider the figure below depicting a valley where there is gentle slope along the y -direction and a U-shape in the x -direction.
- The gradient descent method will bounce violently along the steep sides of the valley while not making much progress along the x -axis.

DIFFICULT TOPOLOGIES - VALLEYS

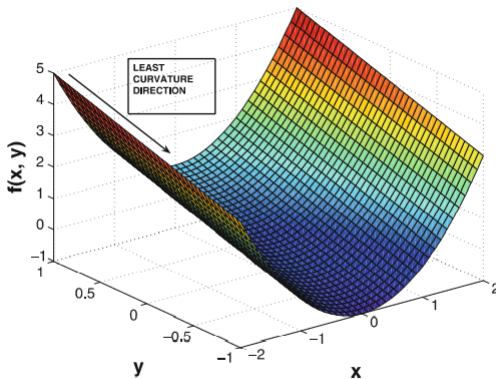


Figure 5.4: The curvature effect in valleys

ADJUSTING FIRST-ORDER DERIVATIVES FOR DESCENT

- We need to find ways of magnifying movement along consistent directions of the gradient to avoid bouncing around the optimal solution.
- We could use second-order information to modify the first-order derivatives by taking curvature into account as we modify components of the gradient. Obtaining second-order information is computationally expensive.
- A computationally less expensive way to handle the problem is to use different learning rates for different parameters because parameters with large partial derivatives show oscillatory behaviour whereas those with small partial derivatives show consistent behaviour.

GRADIENT DESCENT WITH MOMENTUM



- Gradient descent may be slow if the curvature of the function is such that the gradient descent steps hop between the walls of the valley of contours and approaches the optimum slowly.
- If we endow the optimization procedure with memory, we can improve convergence.
- We use an additional term in the step-update to remember what happened in the previous iteration, so that we can dampen oscillations and speed up convergence. This is a momentum term - the name momentum comes from a comparison to a rolling ball whose direction becomes more and more difficult to change as its velocity increases.

MOMENTUM-BASED LEARNING



- Momentum-based methods attack the issues of flat-regions, cliffs and valleys by emphasizing medium-term to long-term directions of consistent movement.
- An aggregated measure of feedback is used to reinforce movement along certain directions and speed up gradient descent.
- The concept of momentum can be illustrated by a marble rolling down a hill that has a number of "local" distortions like potholes, ditches etc. The momentum of the marble causes it to navigate local distortions and emerge out of them.
- The normal update procedure for gradient descent can be written as $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$ where $\mathbf{v} \leftarrow -\alpha \frac{\partial J}{\partial \mathbf{w}}$.

- The momentum-based method remembers the update $\Delta \mathbf{x}_i$ at each iteration i and determines the next update as a linear combination of the current and previous gradients.
- In terms of equations we have
$$\mathbf{x}_{i+1} = \mathbf{x}_i - \gamma_i ((\nabla f)(\mathbf{x}_i))^T + \mathbf{v}_i$$
where $\mathbf{v}_i = \mathbf{0}$ if $i = 0$ and
$$\mathbf{v}_i = \beta \Delta \mathbf{x}_i = \beta (\mathbf{x}_i - \mathbf{x}_{i-1})$$
 if $i > 0$ and $\beta \in [0, 1]$.
- The momentum term is useful where only approximate gradient is known as the momentum term averages out the noisy estimates of the gradient.
- β is referred to as the momentum parameter or the friction parameter.

MOMENTUM-BASED LEARNING



- The figure below shows how momentum-based learning compares to gradient descent, and how momentum allows it to navigate potholes.

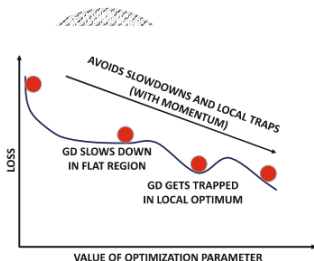


Figure 5.5: Effect of momentum in navigating complex loss surfaces. The annotation “GD” indicates pure gradient descent without momentum. Momentum helps the optimization process retain speed in flat regions of the loss surface and avoid local optima

MOMENTUM-BASED LEARNING



- Momentum-based learning accelerates gradient descent since the algorithm moves quicker in the direction of the optimal solution.
- The useless sideways oscillations as they get cancelled out during the averaging process.
- In the figure on the next slide, it should be clear that momentum increases the relative component of the gradient in the correct direction.

MOMENTUM-BASED LEARNING

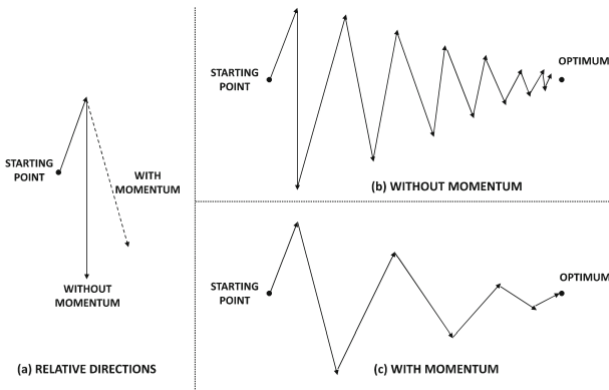


Figure 5.6: Effect of momentum in smoothing zigzag updates

- Here we keep track of the aggregated squared magnitude of the partial derivative with respect to each parameter over the course of the algorithm.
- Mathematically the aggregated squared partial derivative is stored in A_i for the i th variable and we can write
$$A_i \leftarrow A_i + \left(\frac{\partial J}{\partial w_i} \right)^2, \forall i.$$
- The actual update step becomes
$$w_i \leftarrow w_i - \frac{\alpha}{\sqrt{A_i}} \frac{\partial J}{\partial w_i}, \forall i.$$
- Sometimes we might need to avoid ill-conditioning; this is done by adding a small $\epsilon = 10^{-8}$ to A_i in the expression above.



- A_i measures only the historical magnitude of the gradient rather than the sign.
- If the gradient takes values $+100$ and -100 alternatively, A_i will be pretty large and the update step along the parameter in question will be pretty small. On the other hand, if the gradient takes a value 0.1 consistently, A_i will not be as large as before and the update step will be comparatively larger than in the oscillating case.
- The update step along the consistent gradient will be emphasized relative to the update step along the oscillatory component.
- With the passage of time, however, absolute movements along all components will slow down because A_i is monotonically increasing with time.

- AdaGrad suffers from the problem of not making much progress after a while, and the fact that A_i is aggregated over the entire history of partial derivatives which may make the method stale.
- Instead of simply adding squared gradients to estimate A_i , it uses exponential averaging. Therefore the scaling factor A_i does not constantly increase. A_i is updated according to $A_i \leftarrow \rho A_i + (1 - \rho) \left(\frac{\partial J}{\partial w_i} \right)^2$ where $\rho \in (0, 1)$.
- The update step is $w_i \leftarrow w_i - \frac{\alpha}{\sqrt{A_i}} \frac{\partial J}{\partial w_i}, \forall i$.
- The key idea that differentiates RMSPROP from AdaGrad is that the importance of ancient gradients decays exponentially with time as the gradient from t steps before is weighted by ρ^t .

- The Adam method uses similar normalization as AdaGrad and RMSProp.
- It also incorporates momentum into the update and addresses the initialization bias present in RMSProp.
- A_i is exponentially averaged the same way as in RMSProp, i.e $A_i \leftarrow \rho A_i + (1 - \rho) \left(\frac{\partial J}{\partial w_i} \right)^2$ where $\rho \in (0, 1)$.
- An exponentially smoothed gradient for which the i th component is F_i is maintained. Smoothing is performed with a decay parameter ρ_f : $F_i \leftarrow \rho_f F_i + (1 - \rho_f) \frac{\partial J}{\partial w_i}$
- The following update is used at the t th iteration:

$$w_i \leftarrow w_i - \frac{\alpha_t F_i}{\sqrt{A_i}}.$$

- There are two key differences with the RMSProp algorithm - the first difference is the gradient is replaced with the exponentially smoothed value in order to incorporate momentum.

- The second difference is that the learning rate depends on the iteration index t , and is defined as follows:

$$\alpha_t = \alpha \frac{\sqrt{1-\rho^t}}{1-\rho_f^t}.$$

- Both F_t and A_t are initialized to zero which causes bias in early iterations. The two quantities are affected differently which accounts for the equation for α_t .
- As $t \rightarrow \infty$, $\rho^t \rightarrow 0$, $\rho_f^t \rightarrow 0$ and $\alpha_t \rightarrow \alpha$ since $\rho, \rho_f \in (0, 1)$. The default suggested value for ρ_f and ρ are 0.9 and 0.999 respectively.

- There are two key differences with the RMSProp algorithm - the first difference is the gradient is replaced with the exponentially smoothed value in order to incorporate momentum.

- The second difference is that the learning rate depends on the iteration index t , and is defined as follows:

$$\alpha_t = \alpha \frac{\sqrt{1-\rho^t}}{1-\rho_f^t}.$$

- Both F_t and A_t are initialized to zero which causes bias in early iterations. The two quantities are affected differently which accounts for the equation for α_t .
- As $t \rightarrow \infty$, $\rho^t \rightarrow 0$, $\rho_f^t \rightarrow 0$ and $\alpha_t \rightarrow \alpha$ since $\rho, \rho_f \in (0, 1)$. The default suggested value for ρ_f and ρ are 0.9 and 0.999 respectively.

CONSTRAINED OPTIMIZATION AND LAGRANGE MULTIPLIERS

- Consider the following problem: $\min_{\mathbf{x}} f(\mathbf{x})$, $f : \mathbb{R}^D \rightarrow \mathbb{R}$, subject to additional constraints - so we are looking at a minimization problem except that the set of all $\mathbf{x}$ over which minimization is performed is not all of $\mathbb{R}^D$.
- The constrained problem becomes $\min_{\mathbf{x}} f(\mathbf{x})$ subject to $g_i(\mathbf{x}) \leq 0 \forall i=1, 2, \dots, m$.
- Since we have a method of finding a solution to the unconstrained optimization problem, one way to proceed now is to convert the given constrained optimization problem into an unconstrained one.
- We construct $J(\mathbf{x}) = f(\mathbf{x}) + \sum_{i=1}^m \mathbf{1}(g_i(\mathbf{x}))$, where $\mathbf{1}(z) = 0$ for $z \leq 0$ and $\mathbf{1}(z) = \infty$ for $z > 0$.

CONSTRAINED OPTIMIZATION AND LAGRANGE MULTIPLIERS

- The formulation of $J(\mathbf{x})$ in the previous slide ensures that its value is infinity if any one of the constraints $g_i(\mathbf{x})$ is not satisfied. This ensures that the optimal solution to the unconstrained problem is the same as the constrained problem.
- The step-function is also difficult to optimize, and our solution is to replace the step-function by a linear function using Lagrange multipliers.
- We create the Lagrangian of the given constrained optimization problem as follows:
$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_{i=1}^m \lambda_i g_i(\mathbf{x}) = f(\mathbf{x}) + \boldsymbol{\lambda}^T \mathbf{g}(\mathbf{x}), \text{ where } \lambda_i \geq 0 \text{ for all } i.$$

PRIMAL AND DUAL PROBLEMS



- The primal problem is $\min f(\mathbf{x})$ subject to $g_i(\mathbf{x}) \leq 0, 1 \leq i \leq m$. Optimization is performed over the primal variables $\mathbf{x}$.
- The associated Lagrangian dual problem is $\max_{\lambda \in \mathbb{R}^m} \mathcal{D}(\lambda)$ subject to $\lambda \geq 0$ where λ are dual variables.
- $\mathcal{D}(\lambda) = \min_{\mathbf{x} \in \mathbb{R}^d} \mathcal{L}(\mathbf{x}, \lambda)$.
- The following minimax inequality holds over two arguments $\mathbf{x}, \mathbf{y}$: $\max_{\mathbf{y}} \min_{\mathbf{x}} \phi(\mathbf{x}, \mathbf{y}) \leq \min_{\mathbf{x}} \max_{\mathbf{y}} \phi(\mathbf{x}, \mathbf{y})$.

MINIMAX INEQUALITY



- Why is this inequality true?
- Assume that $\mathbf{x}, \mathbf{y}$: $\max_{\mathbf{y}} \min_{\mathbf{x}} \phi(\mathbf{x}, \mathbf{y}) = \phi(\mathbf{x}_A, \mathbf{y}_A)$ and $\min_{\mathbf{x}} \max_{\mathbf{y}} \phi(\mathbf{x}, \mathbf{y}) = \phi(\mathbf{x}_B, \mathbf{y}_B)$.
- Fixing $\mathbf{y}$ at $\mathbf{y}_A$ we see that the inner operation on the left hand side of the minimax inequality is a min operation over $\mathbf{x}$ and returns $\mathbf{x}_A$. Thus we have $\phi(\mathbf{x}_A, \mathbf{y}_A) \leq \phi(\mathbf{x}_B, \mathbf{y}_A)$.
- Fixing $\mathbf{x}$ at $\mathbf{x}_B$ we see that the inner operation on the right hand side of the minimax inequality is a max operation over $\mathbf{y}$ and returns $\mathbf{y}_B$. Thus we have $\phi(\mathbf{x}_B, \mathbf{y}_B) \geq \phi(\mathbf{x}_B, \mathbf{y}_A)$.
- From the above we conclude that $\phi(\mathbf{x}_B, \mathbf{y}_B) \geq \phi(\mathbf{x}_A, \mathbf{y}_A)$.

MINIMAX INEQUALITY



- The difference between $J(\mathbf{x})$ and the Lagrangian $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$ is that the indicator function is relaxed to a linear function.
- When $\boldsymbol{\lambda} \geq 0$, the Lagrangian $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$ is a lower bound on $J(\mathbf{x})$.
- The maximum of $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$ with respect to $\boldsymbol{\lambda}$ is $J(\mathbf{x})$ - if the point $\mathbf{x}$ satisfies all the constraints $g_i(\mathbf{x}) \leq 0$, then the maximum of the Lagrangian is obtained at $\boldsymbol{\lambda} = 0$ and it is equal to $J(\mathbf{x})$. If one or more constraints is violated such that $g_i(\mathbf{x}) > 0$, then the associated Lagrangian coefficient λ_i can be taken to be ∞ so as to equal $J(\mathbf{x})$.

MINIMAX INEQUALITY



- From the previous slide, we have $J(\mathbf{x}) = \max_{\lambda \geq 0} \mathcal{L}(\mathbf{x}, \lambda)$.
- Our original constrained optimization problem boiled down to minimizing $J(\mathbf{x})$, in other words we are looking at $\min_{\mathbf{x} \in \mathbb{R}^d} \max_{\lambda \geq 0} \mathcal{L}(\mathbf{x}, \lambda)$.
- Using the minimax inequality we see that $\min_{\mathbf{x} \in \mathbb{R}^d} \max_{\lambda \geq 0} \mathcal{L}(\mathbf{x}, \lambda) \geq \max_{\lambda \geq 0} \min_{\mathbf{x} \in \mathbb{R}^d} \mathcal{L}(\mathbf{x}, \lambda)$.
- This is known as weak duality. The inner part of the right hand side of the inequality is $\mathfrak{D}(\lambda)$, and the inequality above is the reason for setting up the associated Lagrangian dual problem for the original constrained optimization problem.

MODELING EQUALITY CONSTRAINTS



- Suppose the problem is $\min_{\mathbf{x}} f(\mathbf{x})$ subject to $g_i(\mathbf{x}) \leq 0$ for all $1 \leq i \leq m$ and $h_j(\mathbf{x}) = 0$ for $1 \leq j \leq n$.
- We model the equality constraint $h_j(\mathbf{x}) = 0$ with two inequality constraints $h_j(\mathbf{x}) \geq 0$ and $h_j(\mathbf{x}) \leq 0$.
- The resulting Lagrange multipliers are then unconstrained.
- The Lagrange multipliers for the original inequality constraints are non-negative while those corresponding to the equality constraints are unconstrained.

CONVEX OPTIMIZATION



- We are interested in a class of optimization problems where we can guarantee global optimality.
- When $f()$, the objective function, is a convex function and $g()$ and $h()$ are convex functions, we have a convex optimization problem.
- In this setting we have strong duality - the optimal solution of the primal problem is equal to the optimal solution of the dual problem.
- What is a convex function?

CONVEX FUNCTION



- First we need to know what is a convex set. A set C is a convex set if for any $x, y \in C$, $\theta x + (1 - \theta)y \in C$.
- For any two points lying in the convex set, a line joining them lies entirely in the convex set.
- Let a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a function whose domain is a convex set C .
- The function is a convex function if for any $\mathbf{x}, \mathbf{y} \in C$,
$$f(\theta \mathbf{x} + (1 - \theta) \mathbf{y}) \leq \theta f(\mathbf{x}) + (1 - \theta) f(\mathbf{y})$$
- Another way of looking at a convex function is to use the gradient: for any two points $\mathbf{x}$ and $\mathbf{y}$, we have
$$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla_{\mathbf{x}} f(\mathbf{x})(\mathbf{y} - \mathbf{x}).$$

LINEAR PROGRAMMING



- Let us look at a convex optimization problem where the objective function and constraints are all linear.
- Such a convex optimization problem is called a linear programming problem.
- We can express a linear programming problem as $\min_{\mathbf{x}} \mathbf{c}^T \mathbf{x}$ subject to $\mathbf{Ax} \leq \mathbf{b}$ where $\mathbf{A} \in \mathbb{R}^{m \times d}$ and $\mathbf{b} \in \mathbb{R}^{m \times 1}$.
- The Lagrangian $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$ is given by $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{Ax} - \mathbf{b})$ where $\boldsymbol{\lambda} \in \mathbb{R}^m$ is the vector of non-negative Lagrangian multipliers.
- We can rewrite the Lagrangian as $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})^T \mathbf{x} - \boldsymbol{\lambda}^T \mathbf{b}$.

LINEAR PROGRAMMING



- Taking the derivative of the Lagrangian with respect to $\mathbf{x}$ and setting it to zero we get $\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda} = 0$.
- Since $\mathcal{D}(\boldsymbol{\lambda}) = \min_{\mathbf{x} \in \mathbb{R}^d} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$, plugging in the above equation gives $\mathcal{D}(\boldsymbol{\lambda}) = -\boldsymbol{\lambda}^T \mathbf{b}$.
- We would like to maximize $\mathcal{D}(\boldsymbol{\lambda})$, subject to the constraint $\boldsymbol{\lambda} \geq 0$.
- Thus we end up with the following problem:

$$\begin{aligned} & \max_{\boldsymbol{\lambda} \in \mathbb{R}^m} -\boldsymbol{\lambda}^T \mathbf{b} \\ & \text{subject to } \mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda} = 0 \\ & \boldsymbol{\lambda} \geq 0 \end{aligned}$$

LINEAR PROGRAMMING



- We can solve the original primal linear program or the dual one - the optimum in each case is the same.
- The primal linear program is in d variables but the dual is in m variables, where m is the number of constraints in the original primal program.
- We choose to solve the primal or dual based on which of m or d is smaller.

QUADRATIC PROGRAMMING



- We now consider the case of a quadratic objective function subject to affine constraints:

$$\min_{\mathbf{x} \in \mathbb{R}^d} \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x} \text{ subject to } \mathbf{A} \mathbf{x} \leq \mathbf{b}$$

- Here $\mathbf{A} \in \mathbb{R}^{m \times d}$, $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{c} \in \mathbb{R}^d$

QUADRATIC PROGRAMMING



- The Lagrangian $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$ is given by $\frac{1}{2}\mathbf{x}^T \mathbf{Q}\mathbf{x} + \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{A}\mathbf{x} - \mathbf{b})$.
- Rearranging the above we have $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = \frac{1}{2}\mathbf{x}^T \mathbf{Q}\mathbf{x} + (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})^T \mathbf{x} - \boldsymbol{\lambda}^T \mathbf{b}$
- Taking the derivative of $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$ and setting it equal to zero gives $\mathbf{Q}\mathbf{x} + (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda}) = 0$.
- If we take $\mathbf{Q}$ to be invertible, we have $\mathbf{x} = -\mathbf{Q}^{-1}(\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})$.
- Plugging this value of $\mathbf{x}$ into $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$ gives us $\mathcal{D}(\boldsymbol{\lambda}) = -\frac{1}{2}(\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})\mathbf{Q}^{-1}(\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda}) - \boldsymbol{\lambda}^T \mathbf{b}$.
- This gives us the dual optimization problem:
 $\max_{\boldsymbol{\lambda} \in \mathbb{R}^m} -\frac{1}{2}(\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})\mathbf{Q}^{-1}(\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda}) - \boldsymbol{\lambda}^T \mathbf{b}$ subject to $\boldsymbol{\lambda} \geq \mathbf{0}$.

LAGRANGIAN FORMULATION



- In contrast to the original formulation $\mathcal{D}(\lambda) = \min_{\mathbf{x} \in \mathbb{R}^d} \mathcal{L}(\mathbf{x}, \lambda)$ is an unconstrained optimization problem for a given value of λ .
- We observe that $\mathcal{D}(\lambda) = \min_{\mathbf{x} \in \mathbb{R}^d} \mathcal{L}(\mathbf{x}, \lambda)$ is a point-wise minimum of affine functions and hence $\mathcal{D}(\lambda)$ is concave even though $f()$ and $g()$ may be nonconvex.
- We have obtained a Lagrangian formulation for a constrained optimization problem where the constraints are inequalities. What happens when some constraints are equalities?



Thank you